

SubTrack

Subscription Management System

Project Report

Course	Python Essentials (Build Your Own Project)
Institution	VIT Bhopal University
Program	B.Tech Computer Science and Engineering (Core)
Student Name	Enayet Ali
Registration Number	26BCE10256
GitHub Repository	https://github.com/lapopenayet24-ui/subtrack-subscription-manager
Submission Date	30-09-2026

Table of Contents

Table of Contents	2
2. Introduction	3
3. Problem Statement.....	4
3.1 Scope.....	4
3.2 Target Users	4
4. Objectives.....	5
5. Functional Requirements.....	6
5.1 Module 1 — Subscription Management (CRUD).....	6
5.2 Module 2 — Financial Summary & Reporting	6
5.3 Module 3 — User Session & Data Persistence	6
5.4 Workflow.....	6
6. Non-Functional Requirements	7
7. System Architecture	8
8. Design Diagrams.....	9
8.1 Process Flow / Workflow Diagram.....	9
8.2 Use Case Diagram	9
8.3 Component Diagram.....	10
8.4 Sequence Diagram — Add Subscription	10
8.5 Database / Storage Design	11
9. Design Decisions & Rationale.....	13
10. Implementation Details	14
11. Screenshots / Results.....	15
11.1 Viewing Subscriptions	15
11.2 Viewing the Financial Summary.....	15
11.3 Adding a Subscription	15
12. Testing Approach.....	16
13. Challenges Faced	17
14. Learnings & Key Takeaways	18
15. Future Enhancements.....	19
16. References.....	20

2. Introduction

Most individuals use a range of recurring digital services such as streaming platforms, cloud storage, online learning platforms, and productivity tools, each of which comes with its unique billing period. Without having a comprehensive overview of all these subscriptions, it becomes difficult for individuals to keep track of their active subscriptions, their billing periods, and their total cost monthly or annually.

SubTrack is a Python console application that provides a solution to this problem either for an individual or a group of individuals using the same computer. With this application, a user can add, remove, and view their subscriptions and get to know the total cost incurred every month and year. The data is stored locally within a JSON file without relying on any external database server or Internet connectivity.

This report discusses the objectives, requirements, design, implementation, and evaluation of the SubTrack project that was created as part of the "Build Your Own Project" assignment of the Python Essentials module of VIT Bhopal.

3. Problem Statement

People usually subscribe to more than one recurring subscriptions, which could include things like streaming services, online storage, and other software and fail to keep track of all of those. It becomes hard to notice when a subscription needs to be renewed or when a person stops using the service anymore.

3.1 Scope

Subtrack is a command-line Python program that helps a user manage his/her subscriptions in one place by adding, viewing, and deleting them and calculating the total monthly and annual cost of them. This program stores all information locally using a JSON file; thus, there is no database needed at all.

3.2 Target Users

- People looking for an easy way to monitor their spending on monthly and annual subscriptions.
- Budget-friendly customers who seek a better understanding of their ongoing expenditures.

4. Objectives

Based on the above problem statement, SubTrack was created with the below goals in mind:

- The creation of a Python console application, which would enable a user to perform adding, viewing, and deleting operations on subscription entries without having a database server or any other external dependencies involved.
- The provision of financial overview to users, which will include a correct calculation of the total sum taking into account monthly and yearly billing cycles.
- The storing of subscription information reliably using local JSON files to ensure that any add or delete operation performed successfully by a user won't be lost upon next use.
- The keeping of the code base modular and easy to maintain by splitting up the presentation layer (main.py) and business logic layer (services/subscription_manager.py).
- The keeping of the solution lightweight and dependency-free to enable any user to run it via simple text-based console prompts.

5. Functional Requirements

SubTrack is organized into three major functional modules, each with a clear input/output structure:

5.1 Module 1 — Subscription Management (CRUD)

- Create a subscription that includes the subscription name, cost, billing cycle, category, and expiry date.
- Display all the subscriptions that are available in the database for the current user.
- Delete a subscription if it is cancelled.

5.2 Module 2 — Financial Summary & Reporting

- Total monthly subscription expenditure to be calculated and displayed.
- Total yearly subscription expenditure to be calculated and displayed (normalizing monthly expenses x 12 and yearly expense as is).

5.3 Module 3 — User Session & Data Persistence

- The user needs to be identified through his/her username while initializing the app (initialize_app).
- If there is any subscription data available for the user in subscriptions_data.json then load it, otherwise create a new profile.
- Save each add/remove action performed by the user to the JSON file.

5.4 Workflow

The user controls SubTrack using a numbered text menu system that consists of View, Add, Delete, Financial Summary, and Exit. When any one of the options is chosen from the menu, the functionality is performed by the business logic that either reads or writes the user's record from or into the JSON-based database.

6. Non-Functional Requirements

Requirement	How SubTrack Addresses It
Performance	All operations act on a single user's small in-memory list of subscriptions, so menu actions complete instantly; no network calls or heavy computation are involved.
Usability	A simple numbered menu with plain-English prompts (e.g. "Enter cost") keeps the interface approachable for non-technical users, with the full menu re-printed after every action.
Reliability	Data is written to disk after every add/delete so that no confirmed change is lost if the program is closed or the machine restarts.
Maintainability	Logic is split across <code>main.py</code> (presentation) and <code>services/subscription_manager.py</code> (business logic), making each file easy to read, test, and extend independently.
Error Handling	Numeric input (cost) is parsed inside a try/except block; invalid entries produce a clear error message instead of crashing the program.
Resource Efficiency	The application relies solely on Python's built-in <code>json</code> and <code>os</code> modules, keeping the dependency footprint and memory usage minimal.

7. System Architecture

The SubTrack uses a straightforward three-layer architecture, which decouples the way the user interacts with the application from the processing and storing of the subscription data.

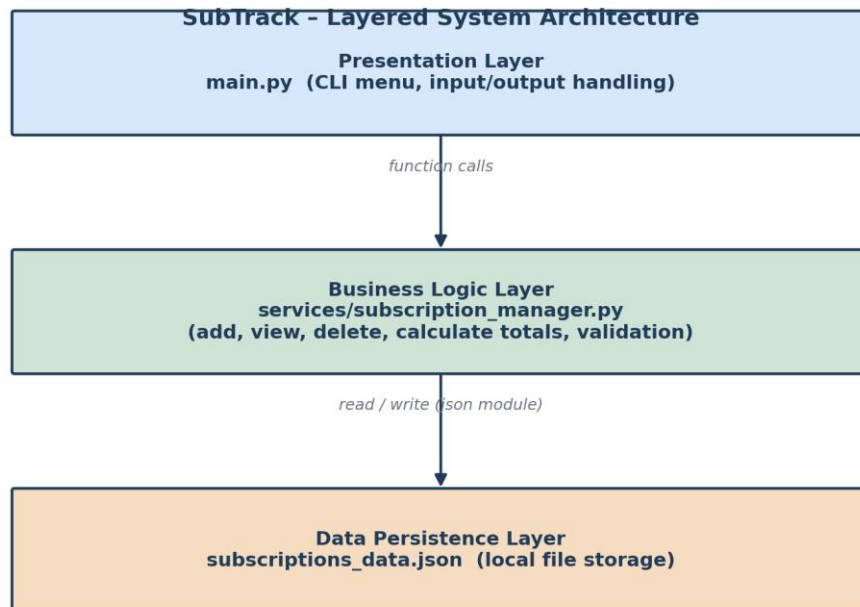


Figure 1: Layered system architecture of SubTrack.

Presentation Layer (main.py)

- Resembles the menu,
- Collects and does basic validation on user input,
- Prints output.

Business Logic Layer (services/subscription_manager.py)

- Contains all functions like `initialize_app()`, `add_new_subscription()`, `view_subscriptions()`, `delete_subscription()` and `calculate_total_expenses()`.

Data Storage Layer (subscriptions_data.json)

- Stores list of subscriptions of all users in JSON file format.

8. Design Diagrams

8.1 Process Flow / Workflow Diagram

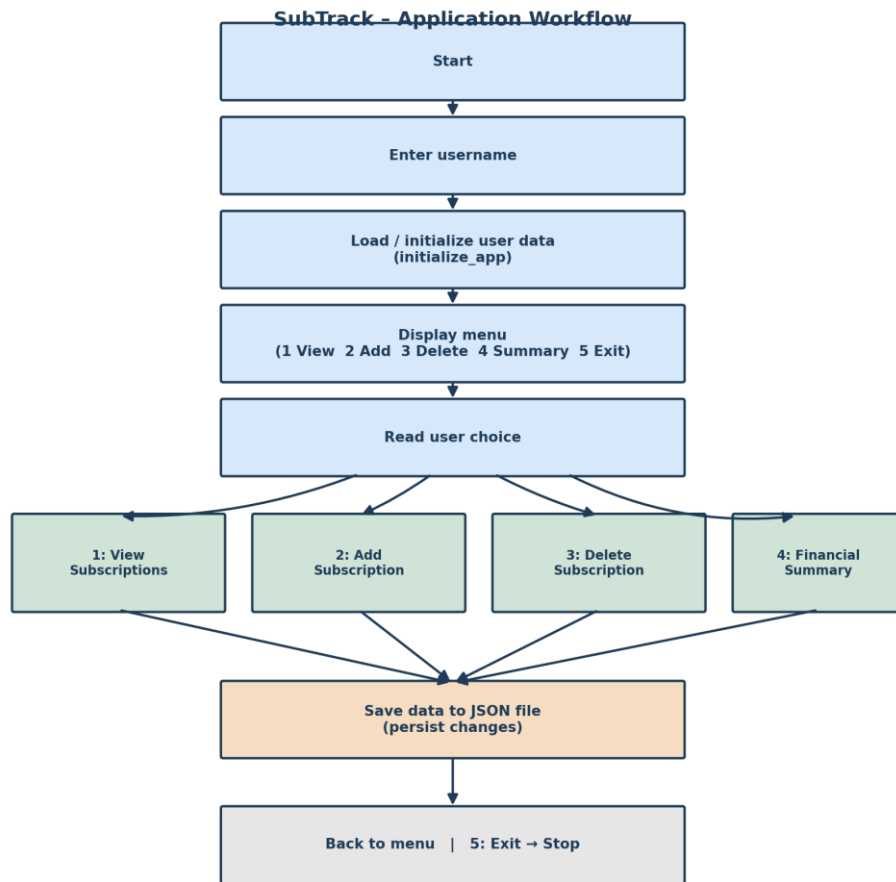


Figure 2: End-to-end workflow from program start to exit.

8.2 Use Case Diagram

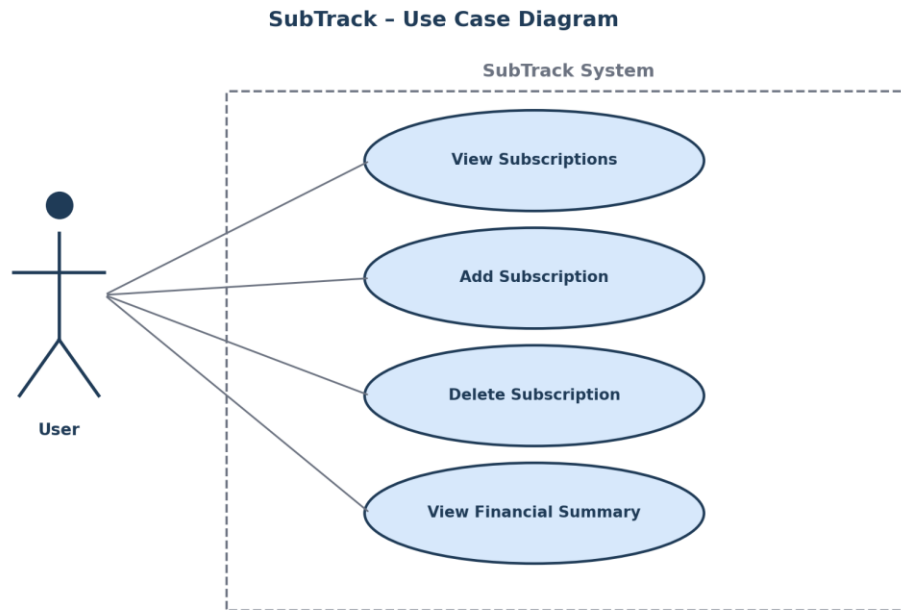


Figure 3: Use cases available to the single actor, the User.

8.3 Component Diagram

The SubTrack system is built using just functions and not classes, hence the use of a component diagram to demonstrate the dependencies between the main.py, subscription_manager.py, and the JSON file as opposed to a class diagram.

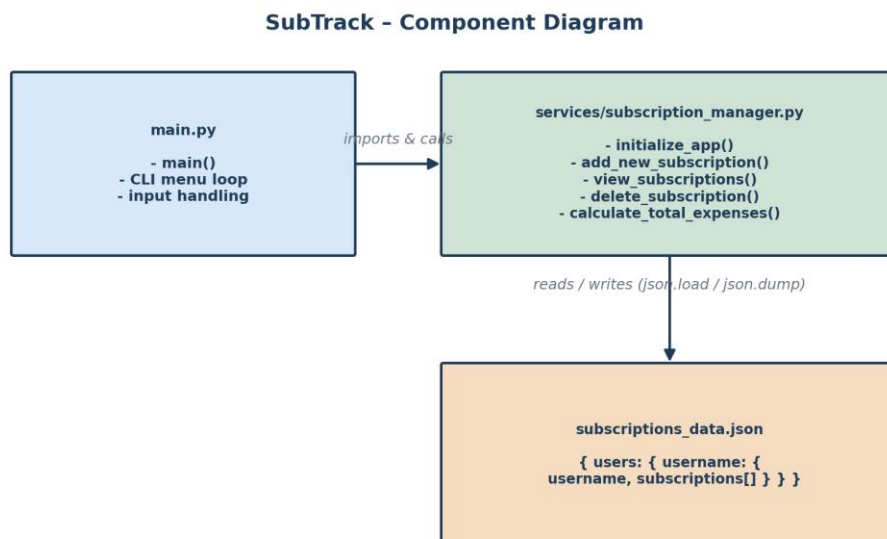


Figure 4: Component-level view of module dependencies.

8.4 Sequence Diagram — Add Subscription

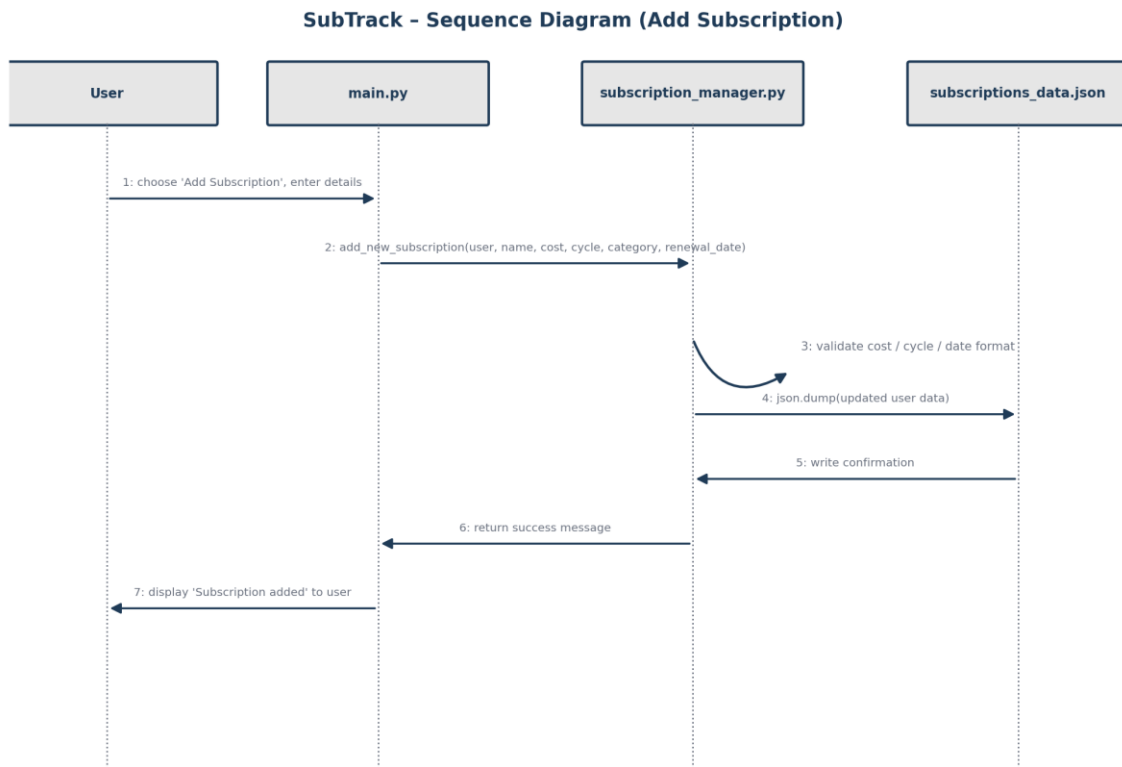


Figure 5: Sequence of calls when a user adds a new subscription.

8.5 Database / Storage Design

ER Diagram: Not applicable. SubTrack uses JSON files to store information instead of relational databases, so no additional tables and foreign keys are required to represent the database structure using an ERD. The “database schema” is the nesting of information in a single JSON file.

Schema Design: The document structure is outlined in the schema below, where each user object contains an array of subscriptions.

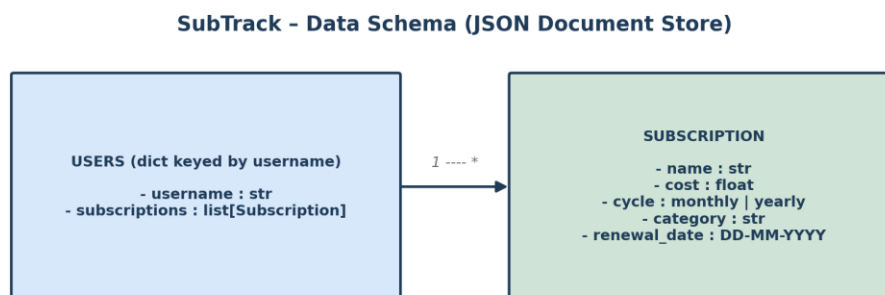


Figure 6: Structure of subscriptions_data.json.

Field	Type	Notes
username	string	Unique key identifying the user record
subscriptions	list	Zero or more Subscription objects
name	string	Subscription / service name, e.g. "Netflix"
cost	float	Billed amount in the user's local currency
cycle	string	"monthly" or "yearly"
category	string	e.g. "Entertainment", "Work"
renewal_date	string (DD-MM-YYYY)	Next renewal / billing date

9. Design Decisions & Rationale

- Menu-driven CLI over a GUI: the brief for the project discourages GUIs and mandates that the project should be able to be executed from the CLI, hence the choice of a numbered text menu.
- JSON over relational database: the subscription data is small and per-user and hierarchic (subscriptions belong to a user), hence making it easy to store them in a JSON object without having to configure any database management system.
- Migration of business logic to `services/subscription_manager.py`: having the I/O and the menu-related code in `main.py` and the business logic in another file allows better modularity, testing, and extendability of the code (such as if we want to implement searching functionality at a later stage).
- Function-based implementation over class-based one: given the constraints of the project in terms of complexity, having a small number of functions rather than a class hierarchy was considered simple enough to develop and test while fulfilling the "5–10 meaningful modules/files" guideline by means of `main.py`, the `services` package, and the data file.
- Defensive parsing of input parameters: cost is parsed inside a `try/except ValueError` structure in `main.py` in order to gracefully handle invalid non-numeric entries and prompt the menu again.

10. Implementation Details

The starting point, `main.py`, asks the user for his name and passes his name to `initialize_app(name)`. It then proceeds into an infinite loop that displays a menu of options and calls the appropriate function in the `services/subscription_manager.py` file based on the user's choice.

Function	Called From	Responsibility
<code>initialize_app(username)</code>	<code>main()</code> at start-up	Loads the user's existing record from the JSON store, or creates a new one if the username is not yet present.
<code>view_subscriptions(user)</code>	Menu options 1 and 3	Prints every subscription currently stored for the active user.
<code>add_new_subscription(user, name, cost, cycle, category, renewal_date)</code>	Menu option 2	Appends a new subscription entry to the user's record and persists the updated data to disk.
<code>delete_subscription(user, name)</code>	Menu option 3	Removes the named subscription from the user's record and persists the change.
<code>calculate_total_expenses(user)</code>	Menu option 4	Aggregates monthly and yearly costs across all of the user's subscriptions and prints the totals.

`main.py` itself includes rudimentary input validation, whereby the cost value that is provided during the addition of a new subscription is read using `float()`, wrapped in a `try/except` block catching `ValueError`, which will ensure that an invalid input (e.g., "abc") will be recognized and an appropriate message displayed without stopping the loop.

Technologies used: Python 3.x; built-in `json` and `os` libraries for reading and writing `subscriptions_data.json`; `Git` and `GitHub` for version control. The project does not require any external libraries to be installed.

11. Screenshots / Results

The sample CLI transcripts below illustrate the expected interaction, based on the sample data shipped in subscriptions_data.json.

11.1 Viewing Subscriptions

```
Welcome to SubTrack Subscription Manager
Enter your username: Enayet Ali
--- Menu ---
1. View Subscriptions
2. Add Subscription
3. Delete Subscription
4. View Financial Summary
5. Exit
Enter your choice: 1

1. Netflix          | Rs.299.0   | monthly | Entertainment | Renews 29-09-2026
2. coursera         | Rs.6000.0  | yearly  | Work           | Renews 31-12-2026
```

11.2 Viewing the Financial Summary

```
Enter your choice: 4

--- Financial Summary ---
Total monthly spend : Rs.299.00
Total yearly spend   : Rs.6000.00 + (Rs.299.00 x 12) = Rs.9588.00
```

11.3 Adding a Subscription

```
Enter your choice: 2
Enter subscription name: Spotify
Enter cost : 199
Enter cycle monthly or yearly: monthly
Enter category Entertainment, Work: Entertainment
Enter renewal date DD-MM-YYYY: 15-10-2026

Subscription 'Spotify' added successfully.
```

12. Testing Approach

Testing was carried out manually against the following cases, mirroring the checklist in the repository's README.md:

ID	Test Case	Steps	Expected Result
TC-01	Add a valid subscription	Choose option 2; enter valid name, cost, cycle, category, date	Subscription appears in "View Subscriptions"
TC-02	Reject non-numeric cost	Choose option 2; enter "abc" for cost	Program shows an error and returns to the menu without crashing
TC-03	Delete a subscription	Choose option 3; enter the name of an existing subscription	Subscription no longer appears in the list
TC-04	Financial summary accuracy	Add subscriptions with mixed monthly/yearly cycles; choose option 4	Totals correctly combine monthly $\times$ 12 and yearly amounts
TC-05	Invalid menu choice	Enter "9" at the main menu	Program shows an error and redisplay the menu instead of exiting
TC-06	Data persistence across runs	Add a subscription, exit, relaunch the program with the same username	Previously added subscription is still present

13. Challenges Faced

- Checking for valid inputs (the cost and date format) without any external libraries.
- Determining the calculation for a single "yearly total" that can be made up of monthly and yearly cycles.
- Arranging subscriptions_data.json in such a way that multiple user data sets can exist in the same file.
- Ensuring main.py doesn't have any business logic so that the menu loop remains simple.

14. Learnings & Key Takeaways

- Hands-on experience with Python's file manipulation capabilities as well as its json library for local storage persistence.
- Learned the importance of keeping separate concerns (presentation code: main.py vs. business logic: services/) apart for readability.
- Learned how to define a database schema (nested JSON structure) prior to implementing CRUD operations on it.
- Learned defensive programming – input validation prior to storage.

15. Future Enhancements

- Renewal alerts – mark those whose renewals will take place in the next N days.
- Modify subscription – enable modification of an existing subscription without having to delete and add it again.
- Export to CSV/Excel – for use in spreadsheets.
- Spending analysis by category and simple graphs (using libraries like matplotlib).
- Automatic unit testing of services/subscription_manager.py methods (for instance using pytest).
- Support multiple currencies and customize date format.
- GUI/web-based application (built using a framework such as Flask) on top of the existing service layer after the CLI core becomes stable.

16. References

- Python Software Foundation — json module documentation, <https://docs.python.org/3/library/json.html>
- VITyarthi — "Build Your Own Project": General Project Instructions & Submission Guidelines (course-provided PDF).
- GitHub Docs — About READMEs, <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>